Name: Aditya Nagesh Raorane Class: D15C/ Batch B Roll No: 44

Experiment 2

Aim: Data Visualization / Exploratory Data Analysis using Matplotlib and Seaborn

Introduction

Exploratory Data Analysis (EDA) is a crucial step in the data analysis pipeline. It involves visually and statistically exploring the data to gain insights and understand its underlying patterns, distributions, and relationships. In this section, we will use Matplotlib and Seaborn, two popular Python libraries, to create visualizations that help in uncovering these insights.

Matplotlib: A comprehensive library for creating static, animated, and interactive visualizations in Python.

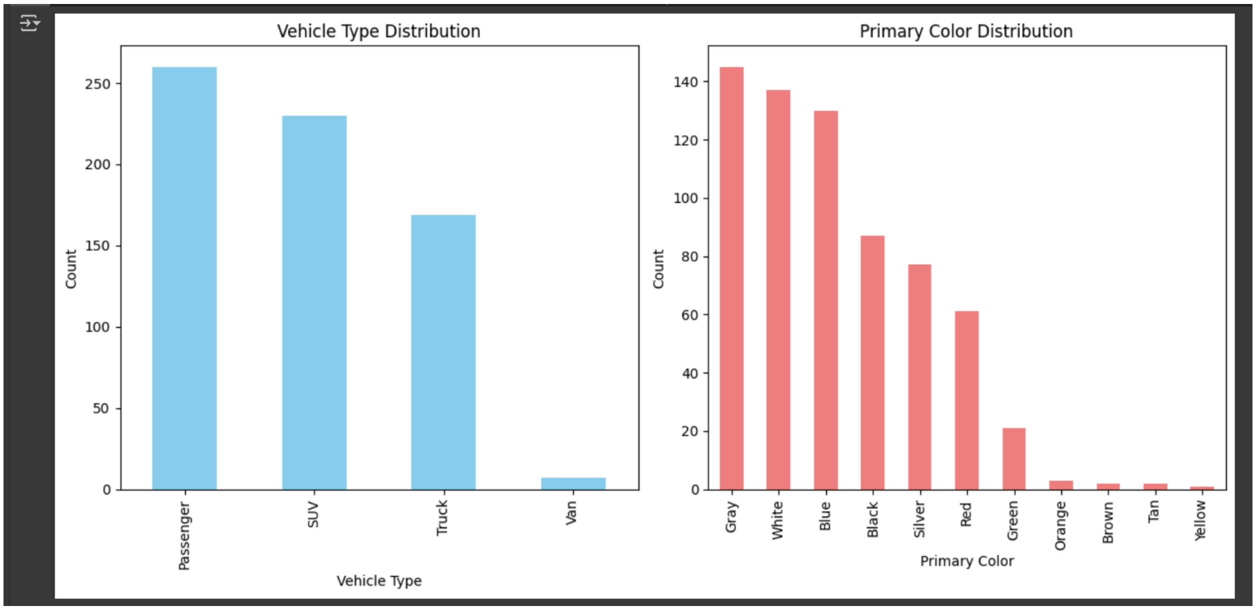
Seaborn: Built on top of Matplotlib, Seaborn provides a high-level interface for drawing attractive and informative statistical graphics.

The primary objective of this analysis is to explore and visualize key patterns in the vehicle dataset, focusing on understanding relationships between different attributes and identifying significant trends.

Steps:

1]We can infer that passengers vehicles with grey and white themed were preferred in most of the regions of US.

```
import matplotlib.pyplot as plt

# Count the occurrences of each category in 'Vehicle Type' and 'Primary Color'
vehicle_type_counts = df['Vehicle Type'].value_counts()
color_counts = df['Primary Color'].value_counts()

# Create bar graph
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
vehicle_type_counts.plot(kind='bar', color='skyblue')
plt.title('Vehicle Type Distribution')
plt.xlabel('Vehicle Type')
plt.ylabel('Count')

plt.subplot(1, 2, 2)
color_counts.plot(kind='bar', color='lightcoral')
plt.title('Primary Color Distribution')
plt.xlabel('Primary Color')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



2] This contingency table shows how many vehicles of each type are purchased in each city.

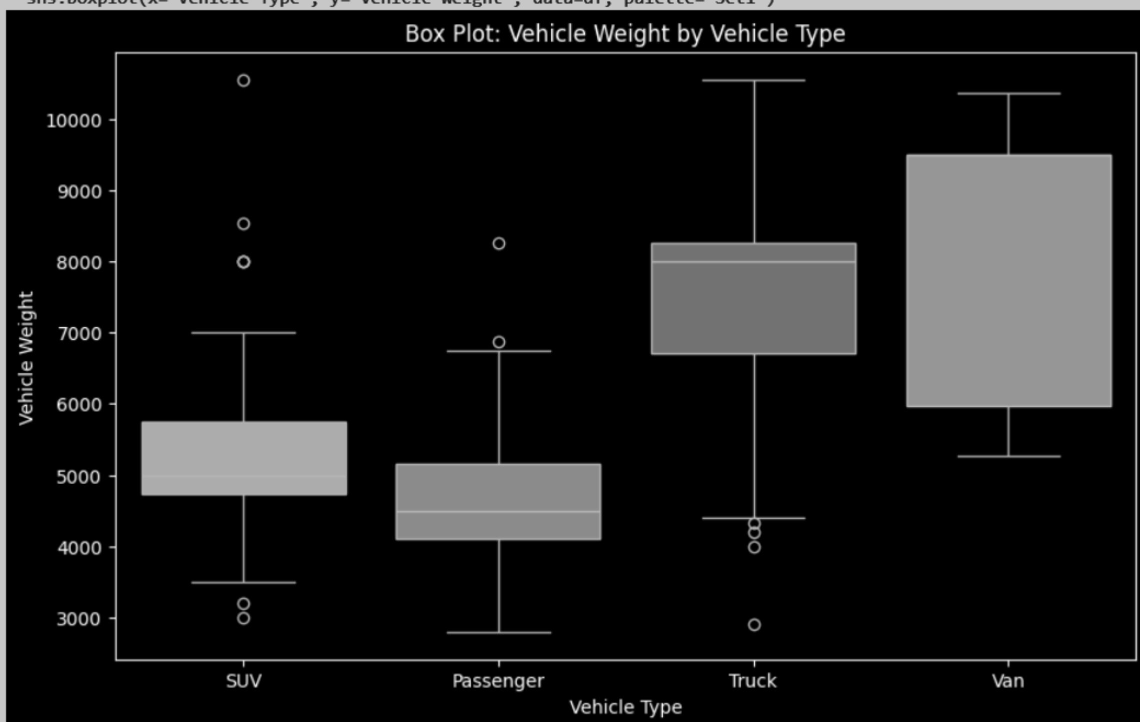
```
[62] # Create the contingency table
     contingency_table = pd.crosstab(df['Primary Customer City'], df['Vehicle Type'])

     # Display the contingency table
     print(contingency_table)
```

```
Vehicle Type           Passenger  SUV  Truck  Van
Primary Customer City
AMSTON                         1    1      0    0
ANDOVER                        1    0      0    0
AVON                           5    4      1    0
BARKHAMSTED                    1    0      0    0
BEACON FALLS                   0    0      1    0
...                          ...  ...    ...  ...
WOLCOTT                        0    2      1    1
WOODBRIDGE                     3    2      3    0
WOODBURY                       0    0      3    0
WOODSTOCK                      2    1      0    0
WOODSTOCK VALLEY               0    1      0    0

[181 rows x 4 columns]
```

3] The plot suggests that vehicle weight varies significantly by type, with Passenger vehicles being the lightest and Vans/Trucks the heaviest. Outliers indicate some extreme-weight vehicles, possibly heavy-duty versions or lightweight modifications.

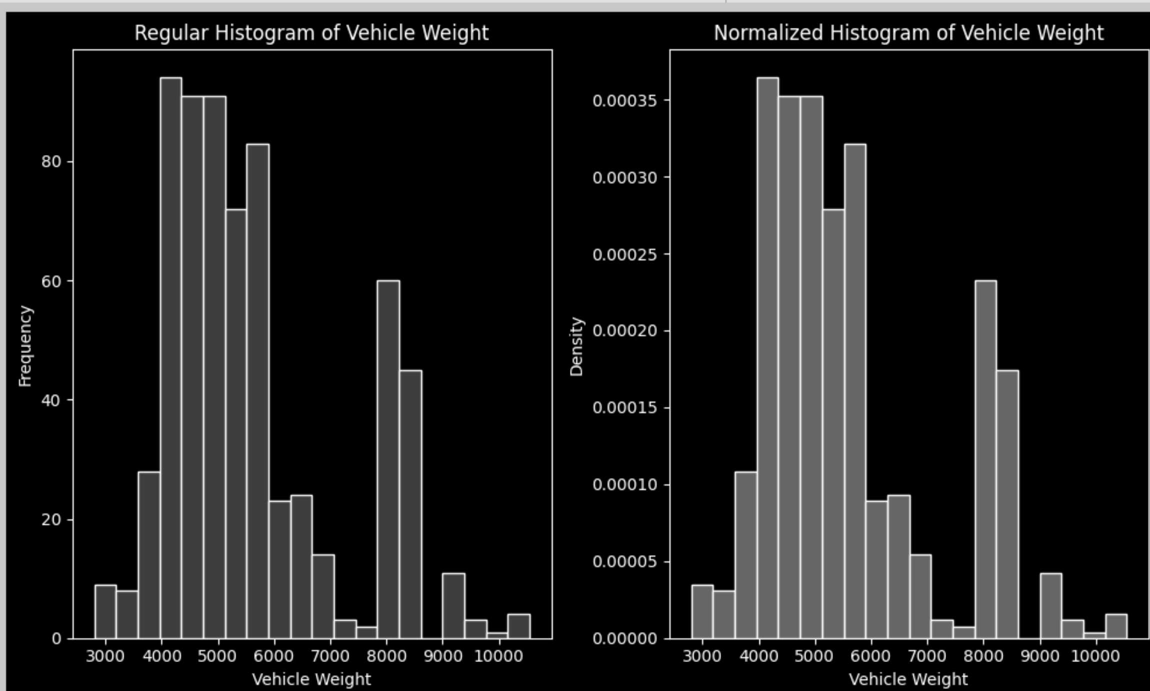
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Vehicle Type', y='Vehicle Weight', data=df, palette='Set1')
plt.title('Box Plot: Vehicle Weight by Vehicle Type')
plt.xlabel('Vehicle Type')
plt.ylabel('Vehicle Weight')
plt.show()
```

```
<ipython-input-42-dd532d0fa891>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to

  sns.boxplot(x='Vehicle Type', y='Vehicle Weight', data=df, palette='Set1')
```



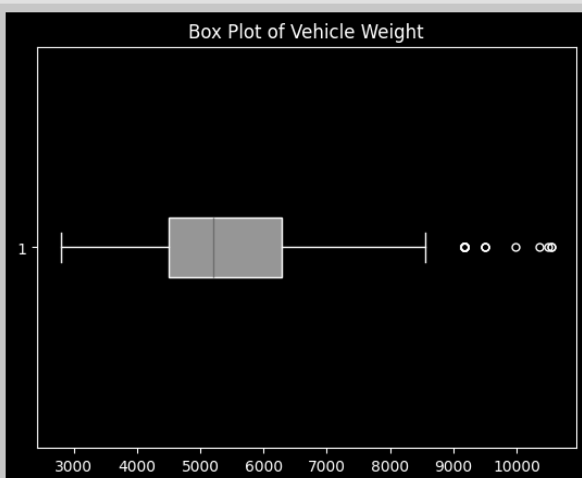
4] The histogram suggests that most vehicles are mid-weight, with a smaller but significant number of heavier vehicles. The bimodal shape indicates two main categories of vehicles, possibly distinguishing passenger vehicles from larger trucks and vans.

```
# Plotting the Regular Histogram
plt.figure(figsize=(10, 6))

# Regular Histogram
plt.subplot(1, 2, 1)
plt.hist(df['Vehicle Weight'], bins=20, color='skyblue', edgecolor='black')
plt.title('Regular Histogram of Vehicle Weight')
plt.xlabel('Vehicle Weight')
plt.ylabel('Frequency')

# Plotting the Normalized Histogram
plt.subplot(1, 2, 2)
plt.hist(df['Vehicle Weight'], bins=20, color='salmon', edgecolor='black', density=True)
plt.title('Normalized Histogram of Vehicle Weight')
plt.xlabel('Vehicle Weight')
plt.ylabel('Density')

# Show both histograms
plt.tight_layout()
plt.show()
```



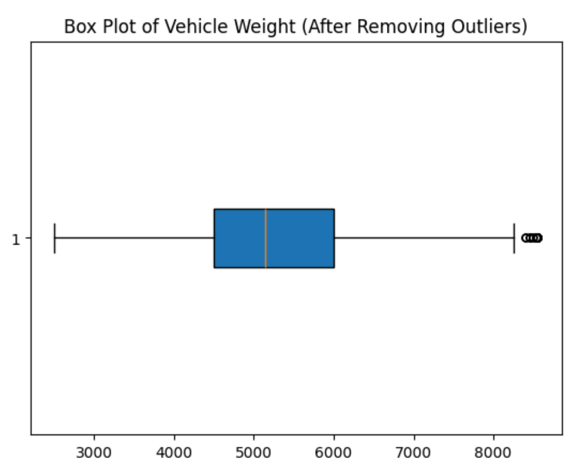
5] The majority of vehicles weigh between 4500 - 7000 lbs, with the median at ~5500 lbs. Some vehicles weigh as little as 3000 lbs, while a few heavy ones exceed 9000 lbs, making them outliers. The distribution is right-skewed, meaning there are more extreme high-weight vehicles than low-weight ones.

```
import pandas as pd
import matplotlib.pyplot as plt

# Plotting the box plot to identify outliers
plt.boxplot(df['Vehicle Weight'], vert=False, patch_artist=True, flierprops=dict(marker='o', color='red', markersize=5))
plt.title('Box Plot of Vehicle Weight')
plt.show()

# Calculate IQR and identify outliers
Q1, Q3 = df['Vehicle Weight'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower_bound, upper_bound = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR

# Filter out outliers
df_no_outliers = df[(df['Vehicle Weight'] >= lower_bound) & (df['Vehicle Weight'] <= upper_bound)]

# Display results
print(f"Original shape: {df.shape}, New shape (no outliers): {df_no_outliers.shape}")
```



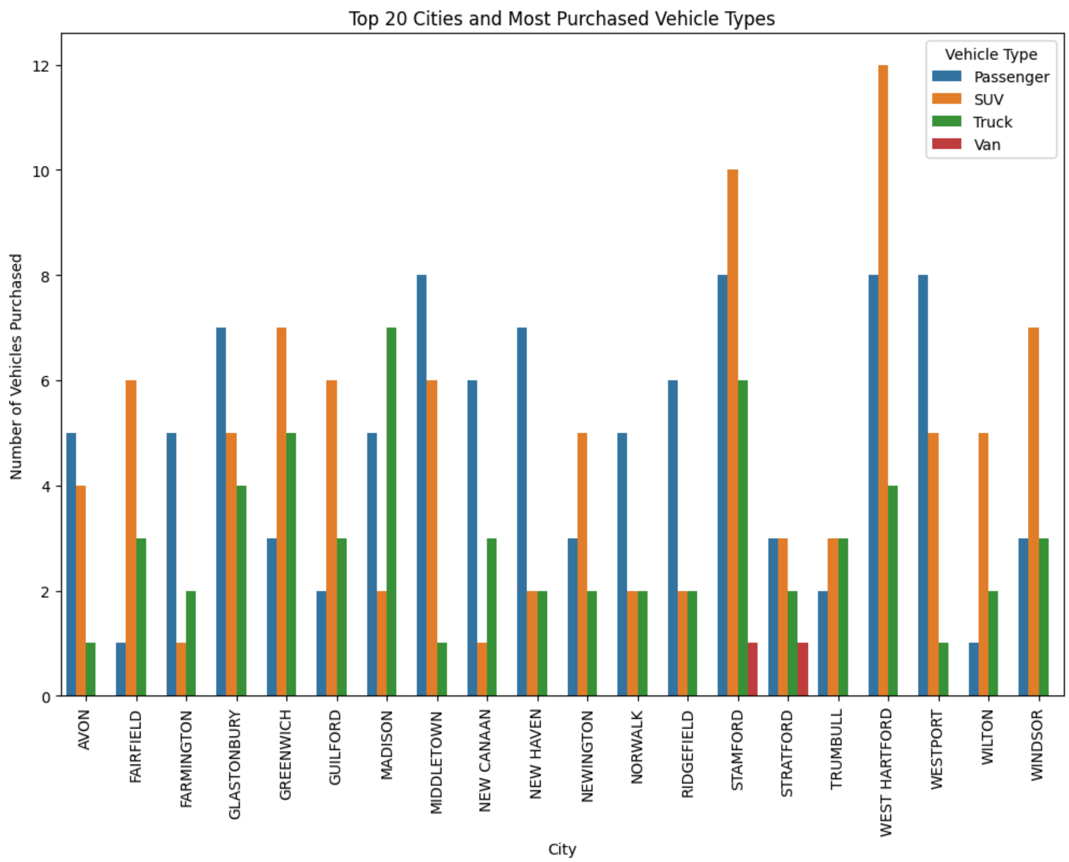

```
Original shape: (666, 10), New shape (no outliers): (647, 10)
```

6] After removing the outliers the new box plot is as follows with the shape as (659, 9) from (678,9).



6] City wise vehicle purchase distribution to understand the geography of the area.

```
# Group by city and vehicle type, then count occurrences
city_vehicle_counts = df.groupby(['Primary Customer City', 'Vehicle Type']).size().reset_index(name='Count')

# Get the top 20 cities based on the number of vehicle purchases
top_20_cities = city_vehicle_counts.groupby('Primary Customer City')['Count'].sum().nlargest(20).index
top_20_data = city_vehicle_counts[city_vehicle_counts['Primary Customer City'].isin(top_20_cities)]

# Plot the data
plt.figure(figsize=(12, 8))
sns.barplot(data=top_20_data, x='Primary Customer City', y='Count', hue='Vehicle Type', dodge=True)
plt.xticks(rotation=90)
plt.title('Top 20 Cities and Most Purchased Vehicle Types')
plt.xlabel('City')
plt.ylabel('Number of Vehicles Purchased')
plt.legend(title='Vehicle Type')
plt.show()
```



Conclusion:

Thus by leveraging Matplotlib and Seaborn, we can gain a deeper understanding of how various vehicle attributes correlate with each other and influence key metrics, such as vehicle registration trends, category distribution, and pricing strategies. The visualizations generated in this EDA can help guide decision-making processes for inventory management, marketing strategies, and customer targeting.